

Lab Activity 6

Objectives

- To design a dashboard and redesign the layout pages.
- To implement and utilize the MUI Component Library
- To revise the code relatively

Materials Needed:

- A computer with Node.js and React JS installed.
- IDE: VS Code, Github Codespaces, or Code Sandbox

Instructions:

Replicate the code snippet.

Follow and implement the enhancement instructions.

Web App Initialization:

Sample design pattern [component based]

```
robles-webprog/
├── README.md
├── project.drawio
├── robles-client/
│   ├── eslint.config.js
│   ├── index.html
│   ├── package-lock.json
│   ├── package.json
│   ├── vite.config.js
│   ├── public/
│   │   └── vite.svg
│   └── src/
│       ├── App.jsx
│       ├── main.jsx
│       ├── assets/
│       │   ├── hero.png
│       │   ├── react.svg
│       │   ├── vite.svg
│       │   └── styles/
│       │       └── index.css
│       ├── components/
│       │   ├── ArticleList.jsx
│       │   ├── Button.jsx
│       │   ├── Footer.jsx
│       │   └── NavBar.jsx
│       ├── data/
│       │   ├── article-content.js
│       │   └── users.json
│       ├── layouts/
│       │   ├── AuthLayout.jsx
│       │   ├── DashLayout.jsx
│       │   └── Layout.jsx
│       └── pages/
│           ├── NotFoundPage.jsx
│           ├── AuthPages/
│           │   ├── SignInPage.jsx
│           │   └── SignUpPage.jsx
│           ├── DashboardPages/
│           │   ├── DashboardPage.jsx
│           │   ├── ReportsPage.jsx
│           │   └── UsersPage.jsx
│           └── LandingPages/
│               ├── AboutPage.jsx
│               ├── ArticleListPage.jsx
│               ├── ArticlePage.jsx
│               └── HomePage.jsx
```



Code Snippets

pages/DashboardPages

ReportsPage.jsx [sample implementation of useRef]

```
import { useRef } from 'react';
import Box from "@mui/material/Box";
import Button from "@mui/material/Button";
import Card from "@mui/material/Card";
import CardContent from "@mui/material/CardContent";
import Stack from "@mui/material/Stack";
import Typography from "@mui/material/Typography";
import { BarChart } from "@mui/x-charts/BarChart";
import { Gauge } from "@mui/x-charts/Gauge";
import { PieChart } from "@mui/x-charts/PieChart";
import { DataGrid } from '@mui/x-data-grid';
const columns = [
  { field: 'id', headerName: 'ID', width: 90 },
  {
    field: 'firstName',
    headerName: 'First name',
    width: 150,
    editable: true,
  },
  {
    field: 'lastName',
    headerName: 'Last name',
    width: 150,
    editable: true,
  },
  {
    field: 'age',
    headerName: 'Age',
    type: 'number',
    width: 110,
    editable: true,
  },
  {
    field: 'fullName',
    headerName: 'Full name',
    description: 'This column has a value getter and is not sortable.',
    sortable: false,
    width: 160,
    valueGetter: (value, row) => `${row.firstName} || '' ${row.lastName} || ''`,
  },
];

const rows = [
  { id: 1, lastName: 'Snow', firstName: 'Jon', age: 14 },
  { id: 2, lastName: 'Lannister', firstName: 'Cersei', age: 31 },
  { id: 3, lastName: 'Lannister', firstName: 'Jaime', age: 31 },
  { id: 4, lastName: 'Stark', firstName: 'Arya', age: 11 },
  { id: 5, lastName: 'Targaryen', firstName: 'Daenerys', age: null },
  { id: 6, lastName: 'Melisandre', firstName: null, age: 150 },
  { id: 7, lastName: 'Clifford', firstName: 'Ferrara', age: 44 },
  { id: 8, lastName: 'Frances', firstName: 'Rossini', age: 36 },
  { id: 9, lastName: 'Roxie', firstName: 'Harvey', age: 65 },
];
```



```
const ReportsPage = () => {
  const printRef = useRef(null);

  const handlePrint = () => {
    const printContent = printRef.current;

    if (!printContent) {
      return;
    }

    const printWindow = window.open('', '_blank', 'width=1200,height=900');

    if (!printWindow) {
      return;
    }

    const headMarkup = Array.from(
      document.querySelectorAll('style, link[rel="stylesheet"]')
    )
      .map((node) => node.outerHTML)
      .join('');

    const exportedAt = new Intl.DateTimeFormat('en-US', {
      dateStyle: 'long',
      timeStyle: 'short',
    }).format(new Date());

    printWindow.document.write(`
      <!DOCTYPE html>
      <html lang="en">
        <head>
          <meta charset="UTF-8" />
          <meta name="viewport" content="width=device-width, initial-scale=1.0" />
          <title>Print Report</title>
          ${headMarkup}
          <style>
            @page {
              size: A4;
              margin: 16mm;
            }

            * {
              box-sizing: border-box;
            }

            body {
              margin: 0;
              font-family: Arial, Helvetica, sans-serif;
              background: #fff;
              color: #1f2937;
            }

            .report-shell {
              padding: 28px;
            }

            .report-header {
              margin-bottom: 24px;
              padding-bottom: 14px;
              border-bottom: 1px solid #d1d5db;
            }

            .report-header h1 {
              margin: 0 0 6px;
              font-size: 28px;
              font-weight: 700;
            }

            .report-header p {
              margin: 0;
              font-size: 14px;
              color: #6b7280;
              line-height: 1.5;
            }

            .report-content .MuiCard-root {
              box-shadow: none !important;
              border: 1px solid #e5e7eb;
              break-inside: avoid;
              page-break-inside: avoid;
            }

            .report-content .MuiCardContent-root {
              padding: 20px;
            }

            .report-content svg {
              max-width: 100%;
            }
          </style>
        </head>
        <body>
          <main class="report-shell">
            <header class="report-header">
              <h1>Reports Summary</h1>
              <p>Analytics overview for generated reports, category breakdown, and
completion performance.</p>
              <p>Prepared on ${exportedAt}</p>
            </header>
            <section class="report-content">
              ${printContent.outerHTML}
            </section>
          </main>
        </body>
      </html>
    `);
    printWindow.document.close();
    printWindow.focus();
    printWindow.print();
  };
};
```



```
return (
  <Box>
    <Stack
      direction={{ xs: "column", md: "row" }}
      justifyContent="space-between"
      alignItems={{ xs: "flex-start", md: "center" }}
      spacing={2}
      sx={{ mb: 4 }}
    >
      <Box>
        <Typography variant="h4" gutterBottom>
          Reports
        </Typography>
        <Typography variant="body1" color="text.secondary">
          Report analytics overview showing generated reports,
          category breakdown, and current completion performance.
        </Typography>
      </Box>

      <Stack direction="row" spacing={1.5} flexWrap="wrap" useFlexGap>
        <Button variant="contained">Generate</Button>
        <Button variant="outlined" onClick={handlePrint}>Export</Button>
        <Button variant="outlined">Filter</Button>
      </Stack>
    </Stack>

    <Stack ref={printRef} spacing={3}>
      <Card>
        <CardContent>
          <Typography variant="h6" gutterBottom>
            Monthly Report Output
          </Typography>
          <Typography variant="body2" color="text.secondary" sx={{ mb: 3 }}>
            This chart compares how many reports were generated and how
            many were completed across the last four months.
          </Typography>
          <BarChart
            series={[
              { data: [18, 24, 20, 27], label: "Generated" },
              { data: [12, 19, 17, 23], label: "Completed" },
            ]}
            height={300}
            xAxis={{
              data: ["January", "February", "March", "April"],
              scaleType: "band",
              label: "Months",
            }},
          />
        </CardContent>
      </Card>

      <Stack direction={{ xs: "column", lg: "row" }} spacing={3}>
        <Card sx={{ flex: 1 }}>
          <CardContent>
            <Typography variant="h6" gutterBottom>
              Report Category Share
            </Typography>
            <Typography
              variant="body2"
              color="text.secondary"
              sx={{ mb: 3 }}
            >
              This chart shows the distribution of report requests by
              category for the current reporting period.
            </Typography>
            <Box sx={{ display: "flex", justifyContent: "center" }}>
              <PieChart
                series={[
                  {
                    data: [
                      { id: 0, value: 14, label: "Sales" },
                      { id: 1, value: 10, label: "Users" },
                      { id: 2, value: 8, label: "Inventory" },
                      { id: 3, value: 6, label: "Finance" },
                    ],
                  },
                ]}
                width={280}
                height={220}
              />
            </Box>
          </CardContent>
        </Card>

        <Card sx={{ flex: 1 }}>
          <CardContent>
            <Typography variant="h6" gutterBottom>
              Completion Rate
            </Typography>
            <Typography
              variant="body2"
              color="text.secondary"
              sx={{ mb: 3 }}
            >
              The gauge highlights the current percentage of reports
              completed on time based on the latest reporting cycle.
            </Typography>
            <Box
              sx={{
                minHeight: 220,
                display: "flex",
                alignItems: "center",
                justifyContent: "center",
              }}
            >
              <Gauge width={180} height={180} value={78} />
            </Box>
          </CardContent>
        </Card>
      </Stack>
    </Card>

    <Card>
      <CardContent>
        <DataGrid
          rows={rows}
          columns={columns}
          experimentalFeatures={{ newEditingApi: true }}
          initialState={{
            pagination: {
              paginationModel: {
                pageSize: 5,
              },
            },
          }}
          pageSizeOptions={[5]}
          checkboxSelection
          disableRowSelectionOnClick
        />
      </CardContent>
    </Card>
  </Box>
);

export default ReportsPage;
```



UsersPage.jsx

```
import { useState } from 'react';
import {
  Alert,
  Box,
  Button,
  Chip,
  Dialog,
  DialogActions,
  DialogContent,
  DialogTitle,
  FormControllLabel,
  IconButton,
  InputAdornment,
  MenuItem,
  Paper,
  Stack,
  Switch,
  TextField,
  Typography,
  useMediaQuery,
} from '@mui/material';
import { useTheme } from '@mui/material/styles';
import Visibility from '@mui/icons-material/Visibility';
import VisibilityOff from '@mui/icons-material/VisibilityOff';
import { DataGrid } from '@mui/x-data-grid';
import usersSeed from '../assets/users.json?raw';

const roles = ['admin', 'editor', 'viewer'];
const genders = ['male', 'female', 'other'];

const blankForm = {
  firstName: '',
  lastName: '',
  age: '',
  gender: '',
  contactNumber: '',
  email: '',
  role: 'editor',
  username: '',
  password: '',
  address: '',
  isActive: true,
};

const labelize = (value) =>
  value ? `${value.charAt(0).toUpperCase()}${value.slice(1)}` : '';

const loadUsers = () => {
  try {
    return {
      users: JSON.parse(usersSeed).map((user, index) => ({
        id: Number(user.id) || index + 1,
        firstName: String(user.firstName ?? '').trim(),
        lastName: String(user.lastName ?? '').trim(),
        age: String(user.age ?? '').trim(),
        gender: genders.includes(String(user.gender ?? '').trim().toLowerCase())
          ? String(user.gender ?? '').trim().toLowerCase()
          : '',
        contactNumber: String(user.contactNumber ?? '').trim(),
        email: String(user.email ?? '').trim().toLowerCase(),
        role: roles.includes(String(user.role ?? '').trim().toLowerCase())
          ? String(user.role ?? '').trim().toLowerCase()
          : 'editor',
        username: String(user.username ?? '').trim().toLowerCase(),
        password: String(user.password ?? ''),
        address: String(user.address ?? '').trim(),
        isActive: typeof user.isActive === 'boolean' ? user.isActive : true,
      })),
      error: '',
    };
  } catch {
    return {
      users: [],
      error: 'Unable to read users from src/assets/users.json.',
    };
  }
};

const seed = loadUsers();
```



```
const UsersPage = () => {
  const theme = useTheme();
  const isMobile = useMediaQuery(theme.breakpoints.down('sm'));
  const [users, setUsers] = useState(seed.users);
  const [modal, setModal] = useState({ open: false, id: null });
  const [form, setForm] = useState(blankForm);
  const [errors, setErrors] = useState({});
  const [showPassword, setShowPassword] = useState(false);

  const resetForm = () => {
    setForm({ ...blankForm });
    setErrors({});
  };

  const openModal = (user) => {
    setModal({ open: true, id: user?.id ?? null });
    setForm(user ? { ...blankForm, ...user } : { ...blankForm });
    setErrors({});
  };

  const closeModal = () => {
    setModal({ open: false, id: null });
    setShowPassword(false);
    resetForm();
  };

  const handleChange = ({ target: { name, value, checked, type } }) => {
    setForm(prev) => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value,
    });

    if (errors[name]) {
      setErrors(prev) => ({ ...prev, [name]: '' });
    }
  };

  const validate = () => {
    const nextErrors = {};
    const email = form.email.trim().toLowerCase();
    const username = form.username.trim().toLowerCase();

    [
      ['firstName', 'First name'],
      ['lastName', 'Last name'],
      ['age', 'Age'],
      ['gender', 'Gender'],
      ['contactNumber', 'Contact number'],
      ['email', 'Email'],
      ['role', 'Role'],
      ['username', 'Username'],
      ['password', 'Password'],
      ['address', 'Address'],
    ].forEach((key, label) => {
      if (!String(form[key]).trim()) {
        nextErrors[key] = `${label} is required.`;
      }
    }));

    if (!nextErrors.email && /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
      nextErrors.email = 'Enter a valid email address.';
    }

    if (!nextErrors.email && users.some(user) => user.id !== modal.id && user.email === email) {
      nextErrors.email = 'Email address already exists.';
    }

    if (!nextErrors.username && users.some(user) => user.id !== modal.id && user.username === username) {
      nextErrors.username = 'Username already exists.';
    }

    return nextErrors;
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    const nextErrors = validate();

    if (Object.keys(nextErrors).length) {
      setErrors(nextErrors);
      return;
    }

    const nextUser = {
      firstName: form.firstName.trim(),
      lastName: form.lastName.trim(),
      age: form.age.trim(),
      gender: form.gender.trim().toLowerCase(),
      contactNumber: form.contactNumber.trim(),
      email: form.email.trim().toLowerCase(),
      role: form.role.trim().toLowerCase(),
      username: form.username.trim().toLowerCase(),
      password: form.password,
      address: form.address.trim(),
      isActive: form.isActive,
    };

    setUsers(prev) =>
      modal.id
        ? prev.map(user => (user.id === modal.id ? { ...user, ...nextUser } : user))
        : [
            ...prev,
            {
              id: prev.reduce((max, user) => Math.max(max, Number(user.id) || 0), 0) + 1,
              ...nextUser,
            },
          ];

    closeModal();
  };

  const toggleStatus = (id) => {
    setUsers(prev) =>
      prev.map(user =>
        user.id === id ? { ...user, isActive: !user.isActive } : user
      );
  };

  const fieldProps = (name, label, extra = {}) => ({
    name,
    label,
    value: form[name],
    onChange: handleChange,
    error: Boolean(errors[name]),
    helperText: errors[name],
    fullWidth: true,
    ...extra,
  });

  const columns = [
    { field: 'id', headerName: 'ID', width: 80 },
    {
      field: 'fullName',
      headerName: 'Full Name',
      flex: 1,
      minWidth: 170,
      valueGetter: (_, row) => `${row.firstName} ${row.lastName}.trim()`,
    },
    { field: 'username', headerName: 'Username', minWidth: 150 },
    { field: 'age', headerName: 'Age', width: 50 },
    {
      field: 'gender',
      headerName: 'Gender',
      minWidth: 110,
      valueGetter: (_, row) => labelize(row.gender),
    },
    {
      field: 'contactNumber', headerName: 'Contact Number', minWidth: 160 },
    { field: 'email', headerName: 'Email', flex: 1.1, minWidth: 220 },
    {
      field: 'role',
      headerName: 'Role',
      minWidth: 120,
      valueGetter: (_, row) => labelize(row.role),
    },
    {
      field: 'status',
      headerName: 'Status',
      minWidth: 120,
      sortable: false,
      renderCell: ({ row }) => (
        <Chip
          size="small"
          label={row.isActive ? 'Active' : 'Inactive'}
          color={row.isActive ? 'success' : 'default'}
          variant={row.isActive ? 'filled' : 'outlined'}
        />
      ),
    },
    {
      field: 'actions',
      headerName: 'Actions',
      minWidth: 220,
      sortable: false,
      filterable: false,
      renderCell: ({ row }) => (
        <Stack direction="row" spacing={1} sx={{ py: 0.5 }}>
          <Button size="small" variant="outlined" onClick={() => openModal(row)}>
            Edit
          </Button>
          <Button
            size="small"
            variant="contained"
            color={row.isActive ? 'warning' : 'success'}
            onClick={() => toggleStatus(row.id)}
          >
            {row.isActive ? 'Disable' : 'Activate'}
          </Button>
        </Stack>
      ),
    },
  ],
};
```



```
return (
  <Box sx={{ width: '100%', minWidth: 0 }}>
    <Box
      sx={{
        mb: 3,
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center',
        gap: 2,
        flexWrap: 'wrap',
      }}
    >
      <Typography variant="h4">Users</Typography>
      <Button
        variant="contained"
        onClick={() => openModal()}
        sx={{ width: { xs: '100%', sm: 'auto' } }}
      >
        Add User
      </Button>
    </Box>

    {seed.error ? (
      <Alert severity="error" sx={{ mb: 2 }}>
        {seed.error}
      </Alert>
    ) : null}

    <Paper sx={{ p: { xs: 1.5, sm: 2 }, minWidth: 0, overflow: 'hidden' }}>
      {users.length ? (
        <Box sx={{ height: { xs: 460, sm: 520 }, width: '100%', minWidth: 0 }}>
          <DataGrid
            rows={users}
            columns={columns}
            disableRowSelectionOnClick
            pageSizeOptions={[5, 10]}
            initialState={{
              pagination: { paginationModel: { pageSize: 5, page: 0 } },
            }}
            sx={{
              minWidth: 0,
              '& .MuiDataGrid-cell, & .MuiDataGrid-columnHeader': {
                outline: 'none',
              },
            }}
          />
        </Box>
      ) : (
        <Alert severity="info">
          No users found. Use Add User to create your first record.
        </Alert>
      )}
    </Paper>

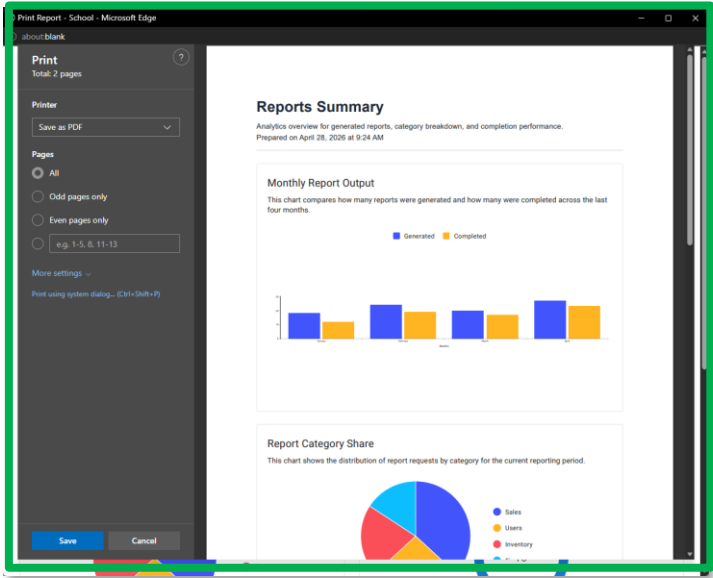
    <Dialog
      open={modal.open}
      onClose={closeModal}
      fullWidth
      fullScreen={isMobile}
      maxWidth="md"
    >
      <Box component="form" onSubmit={handleSubmit}>
        <DialogTitle{modal.id ? 'Edit User' : 'Add User'}</DialogTitle>
        <DialogContent dividers sx={{ px: { xs: 2, sm: 3 } }}>
          <Stack spacing={2} sx={{ pt: 1 }}>
            <Stack direction={{ xs: 'column', sm: 'row' }} spacing={2}>
              <TextField {...fieldProps('firstName', 'First Name')} />
              <TextField {...fieldProps('lastName', 'Last Name')} />
            </Stack>
            <Stack direction={{ xs: 'column', sm: 'row' }} spacing={2}>
              <TextField {...fieldProps('age', 'Age')} />
              <TextField {...fieldProps('gender', 'Gender', { select: true })}>
                {genders.map((gender) => (
                  <MenuItem key={gender} value={gender}>
                    {labelize(gender)}
                  </MenuItem>
                ))}
              </TextField>
            </Stack>
            <Stack direction={{ xs: 'column', sm: 'row' }} spacing={2}>
              <TextField {...fieldProps('contactNumber', 'Contact Number')} />
              <TextField {...fieldProps('email', 'Email Address', { type: 'email' }} />
            </Stack>
            <Stack direction={{ xs: 'column', sm: 'row' }} spacing={2}>
              <TextField {...fieldProps('role', 'Role', { select: true })}>
                {roles.map((role) => (
                  <MenuItem key={role} value={role}>
                    {labelize(role)}
                  </MenuItem>
                ))}
              </TextField>
              <TextField {...fieldProps('username', 'Username')} />
            </Stack>
            <TextField
              {...fieldProps('password', 'Password', {
                type: showPassword ? 'text' : 'password',
                slotProps: {
                  input: {
                    endAdornment: (
                      <InputAdornment position="end">
                        <IconButton
                          edge="end"
                          onClick={() => setShowPassword((prev) =>
                            !prev)}
                        onMouseDown={(event) =>
                          aria-label={showPassword ? 'Hide password' :
                            'Show password'}
                        >
                          {showPassword ? <VisibilityOff /> :
                            <VisibilityOn />}
                        </IconButton>
                      </InputAdornment>
                    )
                  },
                },
              }}
            />
            <TextField {...fieldProps('address', 'Address', { multiline: true, rows:
              3 })} />
            <FormControlLabel
              control={
                <Switch
                  name="isActive"
                  checked={form.isActive}
                  onChange={handleChange}
                />
              }
              label={form.isActive ? 'User status: Active' : 'User status:
                Inactive'}
            />
          </Stack>
          <DialogActions>
            <DialogActions sx={{ px: 3, py: 2 }}>
              <Button onClick={closeModal}>Cancel</Button>
              <Button type="submit" variant="contained">
                {modal.id ? 'Update User' : 'Save User'}
              </Button>
            </DialogActions>
          </Box>
        </Dialog>
      </Box>
    );
  };
export default UsersPage;
```



users.json

```
[
  {
    "firstName": "Alicia",
    "lastName": "Reyes",
    "age": "29",
    "gender": "female",
    "contactNumber": "09171234567",
    "email": "alicia.reyes@robles.dev",
    "role": "admin",
    "username": "aliciareyes",
    "password": "Alicia123!",
    "address": "Sampaloc, Manila, Metro Manila",
    "isActive": true
  },
  {
    "firstName": "Marco",
    "lastName": "Santos",
    "age": "31",
    "gender": "male",
    "contactNumber": "09182345678",
    "email": "marco.santos@robles.dev",
    "role": "viewer",
    "username": "marcosantos",
    "password": "Marco123!",
    "address": "Tondo, Manila, Metro Manila",
    "isActive": true
  },
  {
    "firstName": "Bianca",
    "lastName": "Cruz",
    "age": "26",
    "gender": "female",
    "contactNumber": "09193456789",
    "email": "bianca.cruz@robles.dev",
    "role": "editor",
    "username": "biancacruz",
    "password": "Bianca123!",
    "address": "Quezon City, Metro Manila",
    "isActive": false
  },
  {
    "firstName": "Nathan",
    "lastName": "Diaz",
    "age": "34",
    "gender": "male",
    "contactNumber": "09214567890",
    "email": "nathan.diaz@robles.dev",
    "role": "viewer",
    "username": "nathandiaz",
    "password": "Nathan123!",
    "address": "Pasig City, Metro Manila",
    "isActive": true
  },
  {
    "firstName": "Jasmine",
    "lastName": "Garcia",
    "age": "28",
    "gender": "female",
    "contactNumber": "09225678901",
    "email": "jasmine.garcia@robles.dev",
    "role": "editor",
    "username": "jasminegarcia",
    "password": "Jasmine123!",
    "address": "Makati City, Metro Manila",
    "isActive": false
  },
  {
    "firstName": "Ethan",
    "lastName": "Lopez",
    "age": "33",
    "gender": "male",
    "contactNumber": "09236789012",
    "email": "ethan.lopez@robles.dev",
    "role": "viewer",
    "username": "ethanlopez",
    "password": "Ethan123!",
    "address": "Taguig City, Metro Manila",
    "isActive": true
  }
]
```

Sample Output | Enhancement Instructions:



Users

Search...

LOGOUT

ADD USER

ID	Full Name	Username	Role	Status	Actions
1	Alicia Reyes	aliciareyes	Admin	Active	EDIT DISABLE
2	Marco Santos	marcosantos	Viewer	Active	EDIT DISABLE
3	Blanca Cruz	blancacruz	Editor	Inactive	EDIT ACTIVATE
4	Nathan Diaz	nathandiaz	Viewer	Active	EDIT DISABLE
5	Jasmine Garcia	jasminegarcia	Editor	Inactive	EDIT ACTIVATE

Rows per page: 5

1-5 of 6

Add User

First Name

Last Name

Age

Gender

Contact Number

Email Address

Role

Editor

Password

Address

User status: Active

CANCEL

SAVE USER

- In using MUI:
- Enhancement 1:** Create and design a your printing pdf based on you Laboratory 5 **ReportsPage** design.
- Enhancement 2:** Create and design a search and filter.
- **Add a simple search bar** - Let users search by firstName, lastName, email, or username.
 - **Add basic dropdown filters** - Create simple filters for role, gender, and status (active/inactive)
- Enhancement 3: Improve form validation with beginner-friendly rules**
- Add easy validations such as:

- password must be at least 8 characters
- contact number must be 11 digits
- age must be a number only
- username must not contain spaces

Git Instructions

after the enhancement (must be on root folder surname-webprog)

1. git checkout -b lab-act6
2. git add .
3. git commit -m "lab-act6"
4. git push origin lab-act6

Lab Activity Rubric

Criteria	Excellent (4 pts)	Good (3 pts)	Fair (2 pts)	Poor (1 pt)	Points
Project Setup & Environment	Project is set up correctly with all dependencies installed. The environment is well-configured and ready for development.	Project is set up with minor issues that do not impact core functionality.	Project setup is attempted but has errors affecting development.	Project setup is incomplete or non-functional.	
Navigation & Routing	Navigation is well-structured, intuitive, and includes a responsive design. Routing is properly implemented with dynamic paths if applicable.	Navigation works correctly but may lack responsiveness or design refinements.	Navigation is functional but may have broken links or a confusing structure.	Navigation is missing or not working.	
Component Structure & Reusability	Components are well-structured, reusable, and follow best practices (e.g., props, state, hooks). Code is modular and scalable.	Components are properly used but may lack optimization or modularity.	Components are used but with poor structure, making the code difficult to scale.	Components are poorly implemented, leading to excessive redundancy or poor organization.	
Styling & UI Design	UI is visually appealing, well-structured, and fully responsive. Consistent styling is applied across the project.	UI is functional and mostly styled well but may have minor inconsistencies or responsiveness issues.	UI is present but lacks proper styling, consistency, or responsiveness.	UI is missing or poorly designed, making navigation difficult.	
Content & Functionality	The app displays dynamic content effectively, and all functionalities work as expected without bugs.	Most functionalities work well, but minor issues exist. Content is displayed correctly.	Some functionalities are incomplete, and content may not load properly.	Major functionalities are missing or broken.	
Code Quality & Best Practices	Code follows best practices, is clean, well-commented, and easy to maintain. Proper use of hooks, state, and props is evident.	Code is well-written but may have minor inefficiencies or inconsistent formatting.	Code runs but is cluttered, inefficient, or difficult to read.	Code is disorganized, hard to understand, and does not follow best practices.	
Other Comments/ Observations:				Total Score	
				Rating: (Total Score/24) * 100	